

NPTEL Week 8

ASSIGNMENT 8 SOLUTIONS & EXPLANATIONS

PROGRAMMING SOLUTIONS

1. Elevator Floor Tracker

// PYTHON

```
print(sum(map(int, input().split())), end='')
```

// CONSOLE INJECTION

```
const el = document.querySelector('textarea.inputarea, textarea');
if (el) {
  el.focus();
  document.execCommand('selectAll', false, null);
  document.execCommand('insertText', false, `print(sum(map(int, input().split())), end='')`);
}
```

2. Online Quiz Scoring

// PYTHON

```
score = 0
for x in input().split():
    score += 10 if int(x) == 1 else -2
print(score, end='')
```

// CONSOLE INJECTION

```
const el = document.querySelector('textarea.inputarea, textarea');
if (el) {
  el.focus();
  document.execCommand('selectAll', false, null);
  document.execCommand('insertText', false, `score = 0
for x in input().split():
    score += 10 if int(x) == 1 else -2
print(score, end='')`);
}
```

3. Employee ID Pattern Matching

// PYTHON

```
id1, id2 = input().split()

def is_pattern_match(s1, s2):
    if len(s1) != len(s2):
        return False
    return len(set(s1)) == len(set(s2)) == len(set(zip(s1, s2)))

print(is_pattern_match(id1, id2), end='')
```

// CONSOLE INJECTION

```
const el = document.querySelector('textarea.inputarea, textarea');
if (el) {
    el.focus();
    document.execCommand('selectAll', false, null);
    document.execCommand('insertText', false, `id1, id2 = input().split()

def is_pattern_match(s1, s2):
    if len(s1) != len(s2):
        return False
    return len(set(s1)) == len(set(s2)) == len(set(zip(s1, s2)))

print(is_pattern_match(id1, id2), end='')`);
}
```

MCQ & MSQ SOLUTIONS WITH EXPLANATIONS

Scenario 1: Student Record Management

Q01 Department

Explanation: In Python, indexing starts at 0. Index 2 refers to the third position in the tuple ("Ananya", 1023, "Computer Science", 8.7), which is the department name.

Q02 Because the student details are not expected to change frequently.

Explanation: Tuples are immutable (unchangeable), making them an ideal and memory-efficient choice for static data that shouldn't be accidentally modified.

Q03 4

Explanation: There are four distinct elements in the tuple separated by commas: Name, ID, Department, and CGPA.

Q04 8.7

Explanation: 8.7 is the final element provided at the end of the tuple.

Q05 Create a new tuple with the updated value.

Explanation: Because tuples are immutable, you cannot modify elements directly or append to them. The only way to update the data is to generate a completely new tuple.

Scenario 2: Lucky Draw Betting Game

Q06 7

Explanation: Line 5 is `for i in range(7):`, which dictates that the `for` loop will execute exactly 7 times.

Q07 Lines 6 and 7

Explanation: These are the lines that call `random.randint(1, 10)` to generate both the user's bet and the lucky draw number.

Q08 +₹800

Explanation: Line 12 reads `account = account + 900 - 100`. Mathematically, adding 900 and subtracting 100 results in a net positive increase of 800.

Q09 ₹100 is deducted from the account.

Explanation: The `else` block in Lines 13-14 executes `account = account - 100` when the bet and lucky draw numbers do not match.

Q10 Selected Options: MSQ

—The player's account balance is printed after every round.

—The player's account balance can become negative.

Explanation: The `print()` statement for the account balance is inside the loop, meaning it displays every round. Because the player starts with ₹0 and loses ₹100 on a loss, the balance will dip into the negatives if they lose the first round.

Scenario 3: Secure Access Puzzle Generator

Q11 The displayed puzzle will always have exactly five letters.

Explanation: `random.shuffle()` only rearranges existing elements in a list; it never adds or removes characters. The word "LEVEL" has 5 letters, so the anagram will always have 5 letters.

Q12 `random.shuffle()` arranges the letters in the same order as the original keyword.

Explanation: If the shuffle randomly places the letters into their original spelling, `candidate != keyword` evaluates to `False`, bypassing the `break` statement and forcing the `while` loop to run again.

Q13 A

Explanation: A single-character string cannot be shuffled into a different arrangement. The `candidate != keyword` condition will never be met, causing an infinite loop.

Q14 They stop the loop only after a shuffled word different from the original has been generated.

Explanation: The `if candidate != keyword: break` logic ensures the `while True` loop terminates only when a genuine anagram (different from the input) is formed.

Q15 Selected Options: MSQ

—If the input contains only one character, the program may run indefinitely.

—The generated puzzle always contains exactly the same characters as the original keyword.

Explanation: As proven in Q13, 1-character strings trigger infinite loops. Also, shuffling only moves the provided characters around, guaranteeing the exact same character makeup.

Scenario 4: Customer Feedback Analysis

Q16 To create an object that can analyze the sentiment of text.

Explanation: Line 14 instantiates the `SentimentIntensityAnalyzer()` object from the NLTK VADER library so its methods can be used to score the reviews later.

Q17 compound

Explanation: Line 17 explicitly requests the `["compound"]` key from the dictionary returned by the `polarity_scores()` method to evaluate the overall sentiment.

Q18 It creates a new column containing the sentiment label for each review.

Explanation: `reviews["Sentiment"] = ...` creates a new key/column in the Pandas DataFrame and populates it with the returned values from the applied function.

Q19 It assigns a sentiment label to a single review based on its compound score.

Explanation: The `get_sentiment(review)` function takes a single review string, calculates its compound score, evaluates if it is > 0 , and returns either the string "Positive" or "Negative".

Q20 Selected Options: MSQ

—The reviews are stored in a Pandas DataFrame.

—The sentiment label for each review is generated using the `get_sentiment()` function.

Explanation: Line 4 directly initializes a `pd.DataFrame` to store the reviews. Line 23 uses the `.apply()` method to run the `get_sentiment()` function across the data to generate the labels.